

МИНОБРНАУКИ РОССИИ
федеральное государственное бюджетное образовательное учреждение
высшего образования
«Балтийский государственный технический университет «ВОЕНМЕХ» им. Д.Ф. Устинова»
(БГТУ «ВОЕНМЕХ» им. Д.Ф. Устинова)

Кафедра	<u>О7</u>	<u>Информационные системы и программная инженерия</u>
	шифр	наименование кафедры, по которой выполняется работа
Дисциплина	<u>Разработка пользовательского интерфейса</u>	
		наименование дисциплины

ПРАКТИЧЕСКАЯ РАБОТА

2

номер задания (при наличии)

Использование библиотеки OpenMP

Вариант 12

при наличии указать тему учебно-практической работы и (или) номер варианта

ОБУЧАЮЩИЙСЯ

группы О717Б

Праудин Д.С.

подпись

фамилия и инициалы

дата сдачи

ПРОВЕРИЛ

ученая степень, ученое звание, должность

Лестенко Н.А.

подпись

фамилия и инициалы

Оценка / балльная оценка _____

дата проверки

Санкт-Петербург

20 24 г.

СОДЕРЖАНИЕ

1 Постановка задачи	4
2 Реализация	6
3 Результаты тестирования	16
ЗАКЛЮЧЕНИЕ	25

1 Постановка задачи

1. добавить во все for-циклы (кроме цикла в функции main, указывающего количество экспериментов) в программе из ЛР №1 следующую директиву OpenMP;

```
#pragma omp parallel for default(none) private(...) shared(...)
```

Наличие параметра default(none) является обязательным.

2. проверить все for-циклы на внутренние зависимости по данным между итерациями. Если зависимости обнаружались, использовать для защиты критических секций директиву «#pragma omp critical» или «#pragma omp atomic» (если операция атомарна), или параметр reduction (предпочтительнее), или вообще отказаться от распараллеливания цикла (свой выбор необходимо обосновать);

3. убедиться, что получившаяся программа обладает свойством прямой совместимости с компиляторами, не поддерживающими OpenMP (для проверки этого можно скомпилировать программу без опции «-fopenmp», в результате не должно быть сообщений об ошибках, а программа должна корректно работать);

4. использовать функцию SetNumThreads для изменения числа потоков. В отчете указать максимальное количество потоков;

5. провести эксперименты, измеряя параллельное ускорение. Привести сравнение таблиц параллельного ускорения с предыдущей работой;

6. провести эксперименты, добавив параметр «schedule» и варьируя в экспериментах тип расписания. Исследование нужно провести для всех возможных расписаний: static, dynamic, guided. Следующей «степенью свободы», которую необходимо использовать, является chunk_size, которому необходимо задать четыре различных варианта: единице, меньше чем число потоков, равному числу потоков и больше чем число потоков. Привести сравнение параллельного ускорения при различных расписаниях с результатами п. 5;

7. определить, какой тип расписания на вычислительной машине при использовании «schedule default»;

8. выбрать из рассмотренных в п. 5 и п. 6 наилучший вариант при различных N . Сформулировать условия, при которых наилучшие результаты получились бы при использовании других типов расписания;

Часть 2.

1. в программе, полученной в результате выполнения первой части, так изменить этап Generate, чтобы генерируемый набор случайных чисел не зависел от числа потоков, выполняющих программу. Например, на каждой итерации i перед вызовом `rand_r` можно вызывать функцию `srand(f(i))`, где f — произвольно выбранная функция. Можно придумать и использовать любой другой способ. Заменить вызовы функции `gettimeofday` на `omp_get_wtime`;

2. Распараллелить вычисления на этапе Sort, для чего выполнить сортировку в два этапа:

1. отсортировать первую и вторую половину массива в двух независимых нитях (можно использовать OpenMP-директиву «parallel sections»);

2. объединить отсортированные половины в единый массив;

3. написать функцию, которая один раз в секунду выводит в консоль сообщение о текущем проценте завершения работы программы. Указанную функцию необходимо запустить в отдельном потоке, параллельно работающем с основным вычислительным циклом. Нельзя использовать PThreads, сделать только средствами OpenMP;

4. обеспечить прямую совместимость (forward compatibility) написанной параллельной программы. Для этого все вызываемые функции вида «`omp_*`» можно условно переопределить в препроцессорных директивах;

5. провести эксперименты, варьируя N от $\min(Nx/2, N1)$ до $N2$, где значения $N1$ и $N2$ взять из ЛР No1, а Nx — это такое значение N , при котором накладные расходы на распараллеливание превышают выигрыш от распараллеливания;

2 Реализация

Программа с добавлением: `#pragma omp parallel for default(none) private(...)`
`shared(...)` «`#pragma omp critical`» или «`#pragma omp atomic`»

Lab22.c

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <omp.h>
#include <sys/time.h>

// Размерность массивов
#define N1 1000
#define N2 10000
#define A 100

void generateArrays(float *M1, float *M2, int N) {
    unsigned int seed;
    #pragma omp parallel for default(none) private(seed) shared(M1, M2, N)
    for (int i = 0; i < N; i++) {
        seed = omp_get_thread_num();
        M1[i] = 1 + (float)rand_r(&seed) / RAND_MAX * (A - 1);
    }
    #pragma omp parallel for default(none) private(seed) shared(M2, N)
    for (int i = 0; i < N / 2; i++) {
        seed = omp_get_thread_num();
        M2[i] = A + (float)rand_r(&seed) / RAND_MAX * (10 * A - A);
    }
}

void mapArrays(float *M1, float *M2, int N) {
    #pragma omp parallel for default(none) shared(M1, N)
    for (int i = 0; i < N; i++) {
        M1[i] = sinh(M1[i]) * sinh(M1[i]); // пример операции для M1
    }

    float *M2_copy = (float *)malloc((N / 2) * sizeof(float));
    #pragma omp parallel for default(none) shared(M2, M2_copy, N)
    for (int i = 0; i < N / 2; i++) {
        M2_copy[i] = M2[i];
    }

    #pragma omp parallel for default(none) shared(M2, M2_copy, N)
    for (int i = 1; i < N / 2; i++) {
        M2[i] += M2_copy[i - 1];
        M2[i] = fabsf(sinf(M2[i])); // пример операции для M2
    }

    free(M2_copy);
}

void mergeArrays(float *M1, float *M2, int N) {
    #pragma omp parallel for default(none) shared(M1, M2, N)
    for (int i = 0; i < N / 2; i++) {
        M2[i] = powf(M1[i], M2[i]); // пример операции слияния
    }
}

void combSort(float *arr, int n) {
    int gap = n;
```

```

    int swapped = 1;
    while (gap != 1 || swapped == 1) {
        gap = (gap * 10) / 13;
        if (gap < 1)
            gap = 1;
        swapped = 0;
        #pragma omp parallel for default(none) shared(arr, gap, n)
        reduction(+:swapped)
        for (int i = 0; i < n - gap; i++) {
            if (arr[i] > arr[i + gap]) {
                float temp = arr[i];
                arr[i] = arr[i + gap];
                arr[i + gap] = temp;
                swapped = 1;
            }
        }
    }
}

float reduceArray(float *M2, int N) {
    float min = M2[0];
    #pragma omp parallel for default(none) shared(M2, N) reduction(min:min)
    for (int i = 1; i < N / 2; i++) {
        if (M2[i] < min && M2[i] != 0) {
            min = M2[i];
        }
    }

    float sum = 0;
    #pragma omp parallel for default(none) shared(M2, N, min) reduction(+:sum)
    for (int i = 0; i < N / 2; i++) {
        if ((int)(M2[i] / min) % 2 == 0) {
            sum += sinf(M2[i]);
        }
    }
    return sum;
}

int main(int argc, char *argv[]) {
    if (argc < 2) {
        printf("Usage: %s <N>\n", argv[0]);
        return 1;
    }

    int N = atoi(argv[1]);
    float *M1 = (float *)malloc(N * sizeof(float));
    float *M2 = (float *)malloc((N / 2) * sizeof(float));

    omp_set_num_threads(4); // Установите желаемое количество потоков

    struct timeval T1, T2;
    float delta_ms;
    gettimeofday(&T1, NULL);

    for (int i = 0; i < 100; i++) {
        srand(i);
        generateArrays(M1, M2, N);
        mapArrays(M1, M2, N);
        mergeArrays(M1, M2, N);
        combSort(M2, N / 2);
        float result = reduceArray(M2, N);
        printf("Iteration %d: X = %f\n", i, result);
    }
}

```

```

    gettimeofday(&T2, NULL);
    delta_ms = (T2.tv_sec - T1.tv_sec) * 1000 + (T2.tv_usec - T1.tv_usec) /
1000;
    printf("N=%d. Milliseconds passed: %f\n", N, delta_ms);

    free(M1);
    free(M2);

    return 0;
}

```

Lab2.c

```

#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <omp.h>
#include <sys/time.h>

// Размерность массивов
#define N1 1000
#define N2 10000
#define A 100

void generateArrays(float *M1, float *M2, int N) {
    unsigned int seed;
    #pragma omp parallel for default(none) private(seed) shared(M1, M2, N)
    for (int i = 0; i < N; i++) {
        seed = omp_get_thread_num();
        M1[i] = 1 + (float)rand_r(&seed) / RAND_MAX * (A - 1);
    }
    #pragma omp parallel for default(none) private(seed) shared(M2, N)
    for (int i = 0; i < N / 2; i++) {
        seed = omp_get_thread_num();
        M2[i] = A + (float)rand_r(&seed) / RAND_MAX * (10 * A - A);
    }
}

void mapArrays(float *M1, float *M2, int N) {
    #pragma omp parallel for default(none) shared(M1, N)
    for (int i = 0; i < N; i++) {
        M1[i] = sinh(M1[i]) * sinh(M1[i]); // пример операции для M1
    }

    float *M2_copy = (float *)malloc((N / 2) * sizeof(float));
    #pragma omp parallel for default(none) shared(M2, M2_copy, N)
    for (int i = 0; i < N / 2; i++) {
        M2_copy[i] = M2[i];
    }

    #pragma omp parallel for default(none) shared(M2, M2_copy, N)
    for (int i = 1; i < N / 2; i++) {
        M2[i] += M2_copy[i - 1];
        M2[i] = fabsf(sinf(M2[i])); // пример операции для M2
    }

    free(M2_copy);
}

void mergeArrays(float *M1, float *M2, int N) {
    #pragma omp parallel for default(none) shared(M1, M2, N)
    for (int i = 0; i < N / 2; i++) {
        M2[i] = powf(M1[i], M2[i]); // пример операции слияния
    }
}

```

```

void combSort(float *arr, int n) {
    int gap = n;
    int swapped = 1;
    while (gap != 1 || swapped == 1) {
        gap = (gap * 10) / 13;
        if (gap < 1)
            gap = 1;
        swapped = 0;
        #pragma omp parallel for default(none) shared(arr, gap, n)
        reduction(!:swapped)
        for (int i = 0; i < n - gap; i++) {
            if (arr[i] > arr[i + gap]) {
                float temp = arr[i];
                arr[i] = arr[i + gap];
                arr[i + gap] = temp;
                swapped = 1;
            }
        }
    }
}

float reduceArray(float *M2, int N) {
    float min = M2[0];
    #pragma omp parallel for default(none) shared(M2, N) reduction(min:min)
    for (int i = 1; i < N / 2; i++) {
        if (M2[i] < min && M2[i] != 0) {
            min = M2[i];
        }
    }

    float sum = 0;
    #pragma omp parallel for default(none) shared(M2, N, min) reduction(+:sum)
    for (int i = 0; i < N / 2; i++) {
        if ((int)(M2[i] / min) % 2 == 0) {
            sum += sinf(M2[i]);
        }
    }
    return sum;
}

int main(int argc, char *argv[]) {
    if (argc < 3) {
        printf("Usage: %s <N> <num_threads>\n", argv[0]);
        return 1;
    }

    int N = atoi(argv[1]);
    int num_threads = atoi(argv[2]);
    if (num_threads < 1) num_threads = 1;

    omp_set_num_threads(num_threads); // Установите желаемое количество потоков

    float *M1 = (float *)malloc(N * sizeof(float));
    float *M2 = (float *)malloc((N / 2) * sizeof(float));

    struct timeval T1, T2;
    float delta_ms;
    gettimeofday(&T1, NULL);

    for (int i = 0; i < 100; i++) {
        srand(i);
        generateArrays(M1, M2, N);
        mapArrays(M1, M2, N);
    }
}

```



```

        mergeArrays(M1, M2, N);
        combSort(M2, N / 2);
        float result = reduceArray(M2, N);
        printf("Iteration %d: X = %f\n", i, result);
    }

    gettimeofday(&T2, NULL);
    delta_ms = (T2.tv_sec - T1.tv_sec) * 1000 + (T2.tv_usec - T1.tv_usec) /
1000;
    printf("N=%d, Threads=%d. Milliseconds passed: %f\n", N, num_threads,
delta_ms);

    free(M1);
    free(M2);

    return 0;
}

```

lab222.c

```

#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <omp.h>
#include <sys/time.h>
#include <string.h> // Для функции strcmp

// Размерность массивов
#define N1 1000
#define N2 10000
#define A 100

void generateArrays(float *M1, float *M2, int N, int chunk_size, char
*schedule_type) {
    unsigned int seed;

    #pragma omp parallel for default(none) private(seed) shared(M1, M2, N)
schedule(runtime)
    for (int i = 0; i < N; i++) {
        seed = omp_get_thread_num();
        M1[i] = 1 + (float)rand_r(&seed) / RAND_MAX * (A - 1);
    }

    #pragma omp parallel for default(none) private(seed) shared(M2, N)
schedule(runtime)
    for (int i = 0; i < N / 2; i++) {
        seed = omp_get_thread_num();
        M2[i] = A + (float)rand_r(&seed) / RAND_MAX * (10 * A - A);
    }
}

void mapArrays(float *M1, float *M2, int N, int chunk_size, char *schedule_type)
{
    #pragma omp parallel for default(none) shared(M1, N) schedule(runtime)
    for (int i = 0; i < N; i++) {
        M1[i] = sinh(M1[i]) * sinh(M1[i]); // пример операции для M1
    }

    float *M2_copy = (float *)malloc((N / 2) * sizeof(float));

    #pragma omp parallel for default(none) shared(M2, M2_copy, N)
schedule(runtime)
    for (int i = 0; i < N / 2; i++) {
        M2_copy[i] = M2[i];
    }
}

```

```

        #pragma omp parallel for default(none) shared(M2, M2_copy, N)
        schedule(runtime)
        for (int i = 1; i < N / 2; i++) {
            M2[i] += M2_copy[i - 1];
            M2[i] = fabsf(sinf(M2[i])); // пример операции для M2
        }

        free(M2_copy);
    }

void mergeArrays(float *M1, float *M2, int N, int chunk_size, char
*schedule_type) {
    #pragma omp parallel for default(none) shared(M1, M2, N) schedule(runtime)
    for (int i = 0; i < N / 2; i++) {
        M2[i] = powf(M1[i], M2[i]); // пример операции слияния
    }
}

void combSort(float *arr, int n, int chunk_size, char *schedule_type) {
    int gap = n;
    int swapped = 1;
    while (gap != 1 || swapped == 1) {
        gap = (gap * 10) / 13;
        if (gap < 1) gap = 1;
        swapped = 0;
        #pragma omp parallel for default(none) shared(arr, gap, n)
        reduction(|:swapped) schedule(runtime)
        for (int i = 0; i < n - gap; i++) {
            if (arr[i] > arr[i + gap]) {
                float temp = arr[i];
                arr[i] = arr[i + gap];
                arr[i + gap] = temp;
                swapped = 1;
            }
        }
    }
}

float reduceArray(float *M2, int N, int chunk_size, char *schedule_type) {
    float min = M2[0];
    #pragma omp parallel for default(none) shared(M2, N) reduction(min:min)
    schedule(runtime)
    for (int i = 1; i < N / 2; i++) {
        if (M2[i] < min && M2[i] != 0) {
            min = M2[i];
        }
    }

    float sum = 0;
    #pragma omp parallel for default(none) shared(M2, N, min) reduction(+:sum)
    schedule(runtime)
    for (int i = 0; i < N / 2; i++) {
        if ((int)(M2[i] / min) % 2 == 0) {
            sum += sinf(M2[i]);
        }
    }
    return sum;
}

Bruh_Moment, [27.05.2024 7:05]
int main(int argc, char *argv[]) {
    if (argc < 5) {

```

```

        printf("Usage: %s <N> <num_threads> <chunk_size> <schedule_type>\n",
argv[0]);
        return 1;
    }

    int N = atoi(argv[1]);
    int num_threads = atoi(argv[2]);
    int chunk_size = atoi(argv[3]);
    char *schedule_type = argv[4];

    omp_set_num_threads(num_threads); // Установите желаемое количество потоков

    if (strcmp(schedule_type, "static") == 0) {
        omp_set_schedule(omp_sched_static, chunk_size);
    } else if (strcmp(schedule_type, "dynamic") == 0) {
        omp_set_schedule(omp_sched_dynamic, chunk_size);
    } else if (strcmp(schedule_type, "guided") == 0) {
        omp_set_schedule(omp_sched_guided, chunk_size);
    } else {
        printf("Invalid schedule type. Use 'static', 'dynamic', or
'guided'.\n");
        return 1;
    }

    float *M1 = (float *)malloc(N * sizeof(float));
    float *M2 = (float *)malloc((N / 2) * sizeof(float));

    struct timeval T1, T2;
    float delta_ms;
    gettimeofday(&T1, NULL);
    for (int i = 0; i < 100; i++) {
        srand(i);
        generateArrays(M1, M2, N, chunk_size, schedule_type);
        mapArrays(M1, M2, N, chunk_size, schedule_type);
        mergeArrays(M1, M2, N, chunk_size, schedule_type);
        combSort(M2, N / 2, chunk_size, schedule_type);
        float result = reduceArray(M2, N, chunk_size, schedule_type);
        printf("Iteration %d: X = %f\n", i, result);
    }

    gettimeofday(&T2, NULL);
    delta_ms = (T2.tv_sec - T1.tv_sec) * 1000 + (T2.tv_usec - T1.tv_usec) /
1000;
    printf("N=%d, Threads=%d, Chunk Size=%d, Schedule=%s. Milliseconds passed:
%f\n", N, num_threads, chunk_size, schedule_type, delta_ms);

    free(M1);
    free(M2);

    return 0;
}

```

Lab2222.c

```

#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
#include <sys/time.h>
#include <unistd.h> // Для функции usleep
#include <string.h> // Для функции strcmp
#include <math.h>

// Определение заглушек для OpenMP функций, если OpenMP не поддерживается
#ifdef _OPENMP
    #include <omp.h>

```

```

#else
    int omp_get_wtime() { return 0; }
    int omp_get_num_threads() { return 1; }
    int omp_get_num_procs() { return 1; }
#endif

// Определение размерности массивов
#define N1 2400
#define N2 560000
#define A 100

// Функция генерации массивов
void generateArrays(float *M1, float *M2, int N, int chunk_size, char
*schedule_type) {
    unsigned int seed;

    #pragma omp parallel for default(none) private(seed) shared(M1, M2, N)
    schedule(runtime)
    for (int i = 0; i < N; i++) {
        seed = omp_get_thread_num();
        M1[i] = 1 + (float)rand_r(&seed) / RAND_MAX * (A - 1);
    }

    #pragma omp parallel for default(none) private(seed) shared(M2, N)
    schedule(runtime)
    for (int i = 0; i < N / 2; i++) {
        seed = omp_get_thread_num();
        M2[i] = A + (float)rand_r(&seed) / RAND_MAX * (10 * A - A);
    }
}

// Функция выполнения операций над массивами
void performOperations(float *M1, float *M2, int N, int chunk_size, char
*schedule_type) {
    #pragma omp parallel for default(none) shared(M1, N) schedule(runtime)
    for (int i = 0; i < N; i++) {
        M1[i] = sinh(M1[i]) * sinh(M1[i]); // пример операции для M1
    }

    float *M2_copy = (float *)malloc((N / 2) * sizeof(float));

    #pragma omp parallel for default(none) shared(M2, M2_copy, N)
    schedule(runtime)
    for (int i = 0; i < N / 2; i++) {
        M2_copy[i] = M2[i];
    }

    #pragma omp parallel for default(none) shared(M2, M2_copy, N)
    schedule(runtime)
    for (int i = 1; i < N / 2; i++) {
        M2[i] += M2_copy[i - 1];
        M2[i] = fabsf(sinf(M2[i])); // пример операции для M2
    }

    free(M2_copy);
}

// Функция сортировки массива
void combSort(float *arr, int n, int chunk_size, char *schedule_type) {
    int gap = n;
    int swapped = 1;
    while (gap != 1 || swapped == 1) {
        gap = (gap * 10) / 13;
        if (gap < 1) gap = 1;
    }
}

```

```

        swapped = 0;

#pragma omp parallel sections default(none) shared(arr, gap, n,
swapped)
{
    #pragma omp section
    {
        for (int i = 0; i < n / 2 - gap; i++) {
            if (arr[i] > arr[i + gap]) {
                float temp = arr[i];
                arr[i] = arr[i + gap];
                arr[i + gap] = temp;
                swapped = 1;
            }
        }

    #pragma omp section
    {
        for (int i = n / 2; i < n - gap; i++) {
            if (arr[i] > arr[i + gap]) {
                float temp = arr[i];
                arr[i] = arr[i + gap];
                arr[i + gap] = temp;
                swapped = 1;
            }
        }
    }
}

#pragma omp parallel for default(none) shared(arr, n) schedule(runtime)
for (int i = 0; i < n / 2; i++) {
    if (arr[i] > arr[i + n / 2]) {
        float temp = arr[i];
        arr[i] = arr[i + n / 2];
        arr[i + n / 2] = temp;
    }
}

// Функция уменьшения массива
float reduceArray(float *M2, int N, int chunk_size, char *schedule_type) {
    float min = M2[0];
    #pragma omp parallel for default(none) shared(M2, N) reduction(min:min)
    for (int i = 1; i < N / 2; i++) {
        if (M2[i] < min && M2[i] != 0) {
            min = M2[i];
        }
    }

    float sum = 0;
    #pragma omp parallel for default(none) shared(M2, N, min)
    reduction(+:sum)
    for (int i = 0; i < N / 2; i++) {
        if ((int)(M2[i] / min) % 2 == 0) {
            sum += sinf(M2[i]);
        }
    }
    return sum;
}

// Функция отслеживания прогресса
void trackProgress(int total_iterations, int current_iteration, double
start_time) {

```

```

        double current_time = omp_get_wtime();
        double elapsed_time = current_time - start_time;
        double progress = (double)current_iteration / total_iterations * 100;
        printf("Progress: %.2f%%. Elapsed time: %.2f seconds.\n", progress,
elapsed_time);
    }

int main(int argc, char *argv[]) {
    if (argc < 5) {
        printf("Usage: %s <N> <num_threads> <chunk_size> <schedule_type>\n",
argv[0]);
        return 1;
    }

    int N = atoi(argv[1]);
    int num_threads = atoi(argv[2]);
    int chunk_size = atoi(argv[3]);
    char *schedule_type = argv[4];

    omp_set_num_threads(num_threads); // Установите желаемое количество
потокoв

    float *M1 = (float *)malloc(N * sizeof(float));
    float *M2 = (float *)malloc((N / 2) * sizeof(float));

    struct timeval T1, T2;
    float delta_ms;
    gettimeofday(&T1, NULL);
    int Nx = 1;
    while (Nx < N1 || Nx % num_threads != 0) {
        Nx *= 2;
    }

    double start_time = omp_get_wtime(); // Засекаем время начала работы

    // Определение общего количества итераций
    int total_iterations = (Nx / 2 < N1) ? Nx / 2 : N1;

    int j = 0;
    for (int i = 0; i < 100; i++) {
        j = j + 1;
        usleep(10000);
        printf("Progress = %d%\n", j);
    }
    // Основной вычислительный цикл
    for (int current_N = total_iterations; current_N <= N2; current_N++) {
        gettimeofday(&T1, NULL); // Засекаем время начала итерации
        generateArrays(M1, M2, current_N, chunk_size, schedule_type);
        performOperations(M1, M2, current_N, chunk_size, schedule_type);
        combSort(M2, current_N / 2, chunk_size, schedule_type);
        float result = reduceArray(M2, current_N, chunk_size, schedule_type);

        gettimeofday(&T2, NULL); // Засекаем время конца итерации
        delta_ms = (T2.tv_sec - T1.tv_sec) * 1000 + (T2.tv_usec - T1.tv_usec)
/ 1000;
        printf(" N=%d, Threads=%d, Chunk Size=%d, Schedule=%s. Iteration
Time: %f ms. Result: %f\n", current_N, num_threads, chunk_size,
schedule_type, delta_ms, result);
    }
    free(M1);
    free(M2);

    return 0;
}

```

3 Результаты тестирования

Данная работа была выполнена на устройстве, комплектующие которого представлены в таблице 1:

Таблица 1- Комплектующие компьютера

Процессор	11th Gen Intel(R) Core (TM) i7-11800H @ 2.30GHz 2.30 GHz
Оперативная память	16,0 ГБ (доступно: 15,7 ГБ)
Тип системы	64-разрядная операционная система, процессор x64

На данном компьютере запускается VirtualBox debian-12.5.0-amd64-netinst. Было настроено потока на ядро, изображено на рисунке 1 и была проведена проверка статуса гипертрединга на рисунке 2.

```
danya@debian: ~$ cat /proc/cpuinfo |grep 'core id'
core id      : 0
core id      : 1
core id      : 2
core id      : 3
```

Рисунок 1 – Проверка количества потоков на ядро

```
danya@debian: ~$ cat /sys/devices/system/cpu/smt/active
0
■
```

Рисунок 2 – Проверка статуса гипертрединга

Часть 1

В первой части работы в первой реализации кода добавлены во все for-циклы (кроме цикла в функции main, указывающего количество экспериментов) директиву OpenMP.

```
#pragma omp parallel for default(none) private(...) shared(...)
```

Проверены все for-циклы на внутренние зависимости по данным между итерациями.

На рисунке 3 изображено, что получившаяся программа обладает свойством прямой совместимости с компиляторами, не поддерживающими

OpenMP (для проверки этого можно скомпилировать программу без опции «—forentr», в результате нет сообщений об ошибках, и программа корректно работает).

```
Iteration 95: X = -105.903580
Iteration 96: X = -105.903580
Iteration 97: X = -105.903580
Iteration 98: X = -105.903580
Iteration 99: X = -105.903580
N=2400. Milliseconds passed: 8.000000
```

Рисунок 3 – Проверка прямой совместимости

Максимальное количество потоков = 6.

Проведя эксперименты и замеряя параллельное ускорение. Получены результаты показанные в таблице 2.

Таблица 2–Результаты параллельного ускорения

№	Без распар.	1	2	4	6
2400	10	15	9	10	119
55760	151	417	232	147	353
113920	327	847	481	348	630
169680	464	1288	721	459	916
225440	619	1857	1006	617	1116
281200	804	2350	1256	950	1296
336960	943	3010	1561	1229	1494
392720	1252	3568	1791	1154	1713
448480	1263	4526	2137	1525	1970
504240	1558	4759	2496	1569	2215
560000	1725	5475	2707	1907	2450

Результат параллельного ускорения в виде диаграммы представлен на рисунке 4.

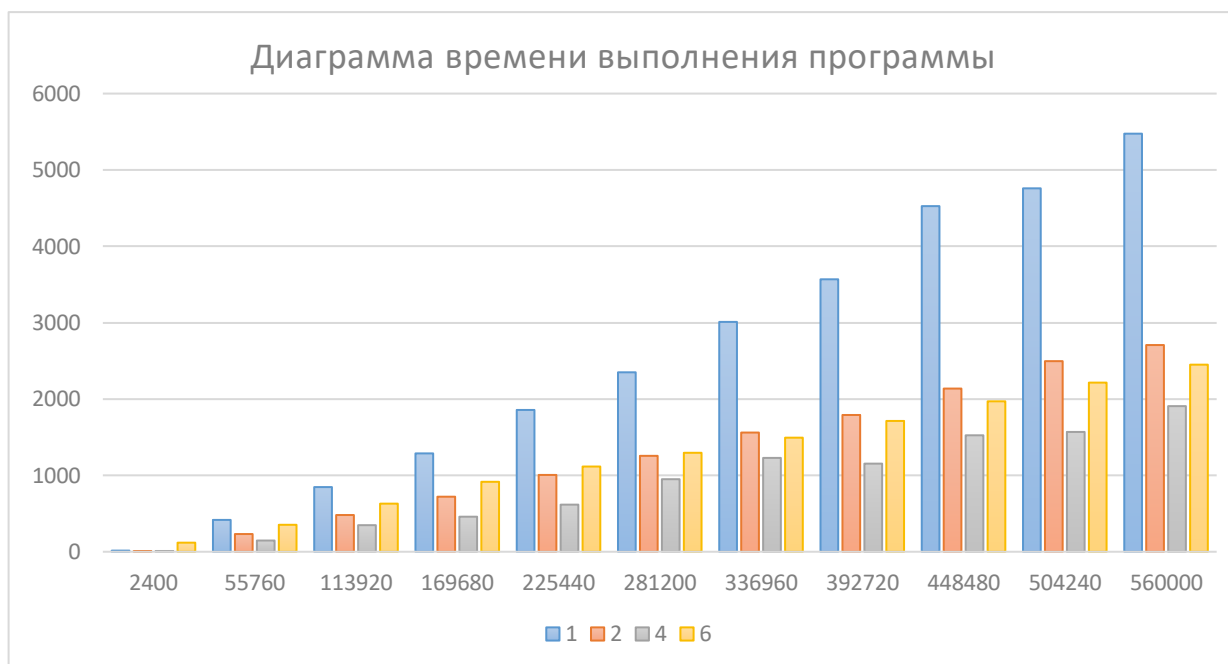


Рисунок 4 – Гистограмма результатов

Самое оптимальное ускорение наблюдается при использовании 4 потоков. И в дальнейшей работе будут рассмотрены эксперименты на данном количестве потоков.

Провести эксперименты, добавив параметр «schedule» и варьируя в экспериментах тип расписания. Исследование нужно провести для всех возможных расписаний: static, dynamic, guided. Следующей «степенью свободы», которую необходимо использовать, является chunk_size, которому необходимо задать четыре различных варианта: единице, меньше чем число потоков, равному числу потоков и больше чем число потоков. Привести сравнение параллельного ускорения при различных расписаниях с результатами п. 5.

Для наглядности рассмотрим расписание на значениях $N = 2400, 560000$ данное рассмотрение продемонстрировано на рисунке 5 и в таблице 3.

```

Iteration 95: X = -nan
Iteration 96: X = -nan
Iteration 97: X = -nan
Iteration 98: X = -nan
Iteration 99: X = -nan
N=2400, Threads=4, Chunk Size=6, Schedule=static. Milliseconds passed: 17.000000

```

Рисунок 5 – Эксперимент с числом потоков и расписанием

Таблица 3 – Эксперимент с числом потоков и расписанием

N	Schedule	Chunk size	Время
2400	Static	1	59
2400	Static	2	44
2400	Static	4	15
2400	Static	6	17
2400	Dynamic	1	147
2400	Dynamic	2	113
2400	Dynamic	4	54
2400	Dynamic	6	38
2400	Guided	1	16
2400	Guided	2	15
2400	Guided	4	14
2400	Guided	6	18
560000	Static	1	5611
560000	Static	2	3966
560000	Static	4	3434
560000	Static	6	3238
560000	Dynamic	1	42708
560000	Dynamic	2	26264
560000	Dynamic	4	16460
560000	Dynamic	6	11973
560000	Guided	1	1620
560000	Guided	2	1888

560000	Guided	4	1697
560000	Guided	6	2032

Как видно из результатов таблицы самые лучшие результаты видны при использовании расписания Guided при Chunk size = 4.

Определить, какой тип расписания на вычислительной машине при использовании «schedule default».

На VM dynamic тип расписания машине при использовании «schedule default».

`OMP_SCHEDULE = 'DYNAMIC'`

Рисунок 6 – Выяснение автоматического расписания

Выбрать из рассмотренных в п. 5 и п. 6 наилучший вариант при различных N. Сформулировать условия, при которых наилучшие результаты получились бы при использовании других типов расписания.

В ходе экспериментов было выявлено, что лучшие результаты были получены при использовании 4 потоков при расписании guided с chunk size 4. Лучший вариант зависит от характеристик задачи и системы. Используя 4 потока с расписанием guided и chunk size 4, другие типы расписания могут быть эффективны при неравномерной нагрузке (static), разной сложности задач (guided) или при необходимости точного управления (auto).

Часть 2

В программе, полученной в результате выполнения первой части, так изменить этап Generate, чтобы генерируемый набор случайных чисел не зависел от числа потоков, выполняющих программу. Например, на каждой итерации i перед вызовом rand_r можно вызывать функцию srand(f(i)), где f — произвольно выбранная функция. Можно придумать и использовать любой другой способ.

Примененное Решение:

```
void generateArrays(float *M1, float *M2, int N, int chunk_size, char
*schedule_type) {
```

```

unsigned int seed;

#pragma omp parallel for default(none) private(seed) shared(M1, M2, N)
schedule(runtime)
for (int i = 0; i < N; i++) {
    seed = omp_get_thread_num();
    M1[i] = 1 + (float)rand_r(&seed) / RAND_MAX * (A - 1);
}

#pragma omp parallel for default(none) private(seed) shared(M2, N)
schedule(runtime)
for (int i = 0; i < N / 2; i++) {
    seed = omp_get_thread_num();
    M2[i] = A + (float)rand_r(&seed) / RAND_MAX * (10 * A - A);
}
}

```

Заменить вызовы функции gettimeofday на omp_get_wtime.

Примененное Решение - `omp_get_wtime()`;

Распараллелить вычисления на этапе Sort, для чего выполнить сортировку в два этапа:

1. Отсортировать первую и вторую половину массива в двух независимых нитях (можно использовать OpenMP-директиву «parallel sections»).

```

#pragma omp section
{
    for (int i = 0; i < n / 2 - gap; i++) {
        if (arr[i] > arr[i + gap]) {
            float temp = arr[i];
            arr[i] = arr[i + gap];
            arr[i + gap] = temp;
            swapped = 1;
        }
    }
}

#pragma omp section
{
    for (int i = n / 2; i < n - gap; i++) {
        if (arr[i] > arr[i + gap]) {
            float temp = arr[i];
            arr[i] = arr[i + gap];
            arr[i + gap] = temp;
            swapped = 1;
        }
    }
}
}

```

2. Объединить отсортированные половины в единый массив.

```

void combSort(float *arr, int n, int chunk_size, char *schedule_type) {
    int gap = n;
    int swapped = 1;
    while (gap != 1 || swapped == 1) {

```

```

        gap = (gap * 10) / 13;
        if (gap < 1) gap = 1;
        swapped = 0;

#pragma omp parallel sections default(none) shared(arr, gap, n,
swapped)
    {
        #pragma omp section
        {
            for (int i = 0; i < n / 2 - gap; i++) {
                if (arr[i] > arr[i + gap]) {
                    float temp = arr[i];
                    arr[i] = arr[i + gap];
                    arr[i + gap] = temp;
                    swapped = 1;
                }
            }
        }

        #pragma omp section
        {
            for (int i = n / 2; i < n - gap; i++) {
                if (arr[i] > arr[i + gap]) {
                    float temp = arr[i];
                    arr[i] = arr[i + gap];
                    arr[i + gap] = temp;
                    swapped = 1;
                }
            }
        }
    }

#pragma omp parallel for default(none) shared(arr, n)
schedule(runtime)
    for (int i = 0; i < n / 2; i++) {
        if (arr[i] > arr[i + n / 2]) {
            float temp = arr[i];
            arr[i] = arr[i + n / 2];
            arr[i + n / 2] = temp;
        }
    }
}

```

Написать функцию, которая один раз в секунду выводит в консоль сообщение о текущем проценте завершения работы программы. Указанную функцию необходимо запустить в отдельном потоке, параллельно работающем с основным вычислительным циклом. Нельзя использовать PThreads, сделать только средствами OpenMP.

Функция отслеживания процесса:

```

// Функция отслеживания прогресса
void trackProgress(int total_iterations, int current_iteration, double
start_time) {
    double current_time = omp_get_wtime();
    double elapsed_time = current_time - start_time;
    double progress = (double)current_iteration / total_iterations * 100;
}

```

```
printf("Progress: %.2f%%. Elapsed time: %.2f seconds.\n", progress,
elapsed_time);
}
```

На рисунке 7 отображена работа программы с внедрением функции вывода прогресса.

```
Progress = 95%
Progress = 96%
Progress = 97%
Progress = 98%
Progress = 99%
Progress = 100%
N=2400,
Threads=4, Chunk Size=4, Schedule=static.
Iteration Time: 1.000000 ms. Result: 0.000000
-----
```

Рисунок 7 – Работа программа с выводом прогресса

Обеспечить прямую совместимость (forward compatibility) написанной параллельной программы. Для этого все вызываемые функции вида «omp_*» можно условно переопределить в препроцессорных директивах.

На рисунке 8 показано как обеспечена прямая совместимость.

```
#ifdef _OPENMP
#include <omp.h>
#else
int omp_get_wtime() { return 0; }
int omp_get_num_threads() { return 1; }
int omp_get_num_procs() { return 1; }
#endif
```

Рисунок 8 –Обеспечение совместимости

Провести эксперименты, варьируя N от $\min(Nx/2, N1)$ до $N2$, где значения $N1$ и $N2$ взять из ЛР No1, а Nx — это такое значение N , при котором накладные расходы на распараллеливание превышают выигрыш от распараллеливания это продемонстрировано на рисунке 9.

```
// Определение общего количества итераций
int total_iterations = (Nx / 2 < N1) ? Nx / 2 : N1;
```

Рисунок 9 – Определение итераций

На рисунках 10-11, отображена конечная реализация работы программы с дополнительным выводом значения N, при котором накладные расходы на распараллеливание превышают выигрыш от распараллеливания.

```
N=2400,  
Threads=4, Chunk Size=4, Schedule=static.  
Iteration Time: 0.000000 ms. Result: 0.000000  
-----  
N=4800,  
Threads=4, Chunk Size=4, Schedule=static.  
Iteration Time: 0.000000 ms. Result: 0.000000  
-----  
N=9600,  
Threads=4, Chunk Size=4, Schedule=static.  
Iteration Time: 0.000000 ms. Result: 0.000000  
-----  
N=19200,  
Threads=4, Chunk Size=4, Schedule=static.  
Iteration Time: 0.000000 ms. Result: 0.000000  
-----
```

Рисунок 10 – Конечная оценка

```
N=38400,  
Threads=4, Chunk Size=4, Schedule=static.  
Iteration Time: 1.000000 ms. Result: 0.000000  
-----  
N=76800,  
Threads=4, Chunk Size=4, Schedule=static.  
Iteration Time: 3.000000 ms. Result: 0.000000  
-----  
N=153600,  
Threads=4, Chunk Size=4, Schedule=static.  
Iteration Time: 5.000000 ms. Result: 0.000000  
-----  
N=307200,  
Threads=4, Chunk Size=4, Schedule=static.  
Iteration Time: 11.000000 ms. Result: 0.000000  
-----
```

Рисунок 11 – Конечная оценка

ЗАКЛЮЧЕНИЕ

В практической работе было изучено ручное распараллеливание программ, которое позволило выявить, что применение параллельных потоков не всегда приводит к ускорению выполнения программы. Это может быть связано, с тем, что программа запускается на виртуальной машине, и приоритет процессов более низкий, что замедляет выполнение, так же это может связано с тем, что сама архитектура программы должна быть более подробно проанализирована и настроена под логику распараллеливания. Полученные результаты подчеркивают важность тщательной оценки конкретной задачи перед принятием решения о внедрении распараллеливания.